Provider Pattern & Composition Pattern

6.5 – Provider Pattern & Composition Pattern

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is the Provider pattern in React?"	🟡🔴	★★★★☆
"What is composition over inheritance?"	🟡🔴	★★★★☆
"How do you use the children prop for composition?"	🟡	★★★☆☆

Provider Pattern

The Provider pattern uses React's Context API to make data available to any component in a subtree without prop drilling. The "provider" wraps a section of the component tree and supplies a value. Any descendant can consume it.

This pattern is used by virtually every React library that needs to share state: React Redux (`<Provider store={store}>`), React Router (`<BrowserRouter>`), Theme providers, Auth providers, and internationalization (i18n) providers.

JSX Copy

```jsx
const ThemeContext = createContext('light');

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme(t => t === 'light' ? 'dark' : 'light'

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) throw new Error('useTheme must be used inside ThemeProvide
  return context;
}

// Wrap the app
<ThemeProvider>
  <App />
</ThemeProvider>

// Use anywhere inside
function Header() {
  const { theme, toggleTheme } = useTheme();
  return (
    <header className={theme}>
      <button onClick={toggleTheme}>Toggle Theme</button>
    </header>
  );
}
```

The custom `useTheme` hook adds a safety check and gives consumers a clean API. This pattern of Provider + custom hook is the standard approach in modern React.

Composition Pattern

Composition in React means building complex UI by combining simpler components, using the `children` prop and named "slot" props instead of inheritance.

Using `children` for flexible content

JSX Copy

```jsx
function Card({ children }) {
  return <div className="card">{children}</div>;
}

// Same Card, completely different content
<Card>
  <h2>User Profile</h2>
  <p>Alice, 28 years old</p>
</Card>

<Card>
  <img src="/product.jpg" alt="Product" />
  <h3>$29.99</h3>
  <button>Add to Cart</button>
</Card>
```

Named slots for multi-section layouts

JSX Copy

```jsx
function Dialog({ title, content, actions }) {
  return (
    <div className="dialog-overlay">
      <div className="dialog">
        <div className="dialog-title">{title}</div>
        <div className="dialog-content">{content}</div>
```

```
  );
}

<Dialog
  title={<h2>Confirm Deletion</h2>}
  content={<p>This action cannot be undone.</p>}
  actions={
    <>
      <button onClick={onCancel}>Cancel</button>
      <button onClick={onConfirm}>Delete</button>
    </>
  }
/>
```

Specialization through composition (not inheritance)

JSX Copy

```
// Base component
function Button({ children, variant = 'default', ...props }) {
  return <button className={`btn btn-${variant}`} {...props}>{children}</b
}

// Specialized versions through composition, not inheritance
function PrimaryButton(props) {
  return <Button variant="primary" {...props} />;
}

function DangerButton(props) {
  return <Button variant="danger" {...props} />;
}

function IconButton({ icon, children, ...props }) {
  return <Button {...props}><span className="icon">{icon}</span> {children
}
```

This is React's alternative to class inheritance. The React team explicitly recommends composition over inheritance for all code reuse scenarios.

Interview answer: "The Provider pattern uses Context to share data across a component tree without prop drilling. I implement it as a Provider component paired with a custom hook (like ThemeProvider + useTheme) for a clean API. The Composition pattern builds complex UI by combining simpler components using the children prop and named slot props. Instead of creating Button subclasses through inheritance, I create specialized components like PrimaryButton and DangerButton that compose the base Button with different props. React explicitly recommends composition over inheritance."

Part of the React Interview Preparation Kit — Back to Course Roadmap →

QUICK CHECK — no marks, just for you.

1. According to the lesson, what problem does the Provider pattern primarily solve?

- ◯ It replaces useState so that components no longer need local state
- ◯ It makes data available to any component in a subtree without prop drilling
- ◯ It prevents child components from re-rendering when their parent re-renders
- ◯ It lets a parent component read state that is defined inside its children

Check answer

2. In the lesson's ThemeProvider example, why is the custom useTheme hook written to throw an error when the context value is missing, rather than having components call useContext(ThemeContext) directly?

- ◯ It adds a safety check for consumers used outside the provider and gives them a cleaner API
- ◯ Throwing forces React to re-render the provider so the theme value is recalculated

Custom hooks are the only way to read a context value created with createContext

- ○ useContext cannot be called from more than one component, so the hook is required to share it

Check answer

3. The lesson defines PrimaryButton and DangerButton on top of a base Button. What technique does it say React uses for this kind of specialisation?

- ○ Composing the base Button with different props, instead of class inheritance
- ○ Extending the Button class and overriding its render method
- ○ Registering variants on the Button component through a static subclass registry
- ○ Cloning Button with React.cloneElement inside a higher-order component

Check answer

‹ Previous
Compound Component Pattern

Next ›
SOLID Principles in React

LEARN MORE

ARTICLES & DOCS

- Passing Data Deeply with Context
  react.dev
- Passing Props (composition & children)
  react.dev

VIDEOS

React.js Interview Kit



The Provider pattern

YouTube



Composition pattern in React

YouTube



Context API & Provider pattern

YouTube

Hand-picked references for this lesson.